

Estudo Dirigido

Testes de Aceitação utilizando *Behave* e *Python*

1 INSTALAÇÃO

Existem implementações do *Cucumber*¹ para diferentes linguagens, como Java, JavaScript, Ruby, Scala e outras. Para o Python, o framework correspondente ao Cucumber mais conhecido é o *Behave*². Neste estudo dirigido será utilizado o framework Behave para Python, juntamente com o IDE PyCharm Professional. Para instalar este framework, siga as seguintes orientações a seguir.

No *Moodle* da disciplina, faça *download* do arquivo *Leilão-Python.zip*. Descompacte o arquivo e crie um projeto *Leilao* no IDE PyCharm (ou em um outro IDE de sua preferência).

No terminal do projeto, instale o framework Behave com o comando "pip install behave", assim como ilustra a Figura 1.

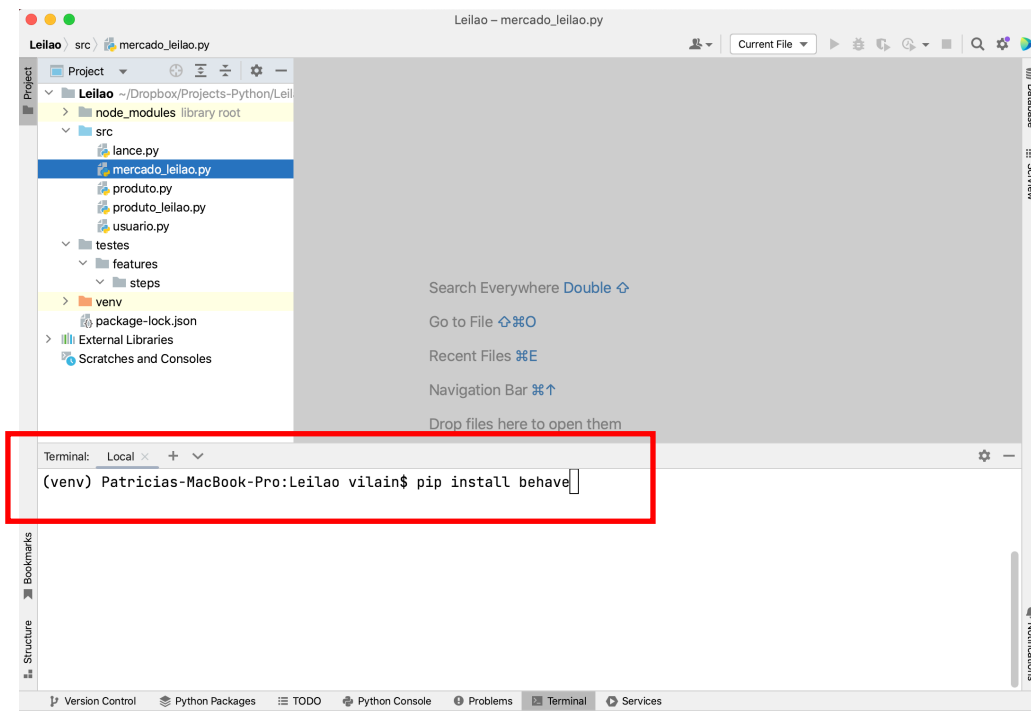


Figura 1 - Instalação do Behave.

¹ <https://cucumber.io/>

² <https://behave.readthedocs.io/>

O framework *Behave* será instalado no projeto corrente (Figura 2).

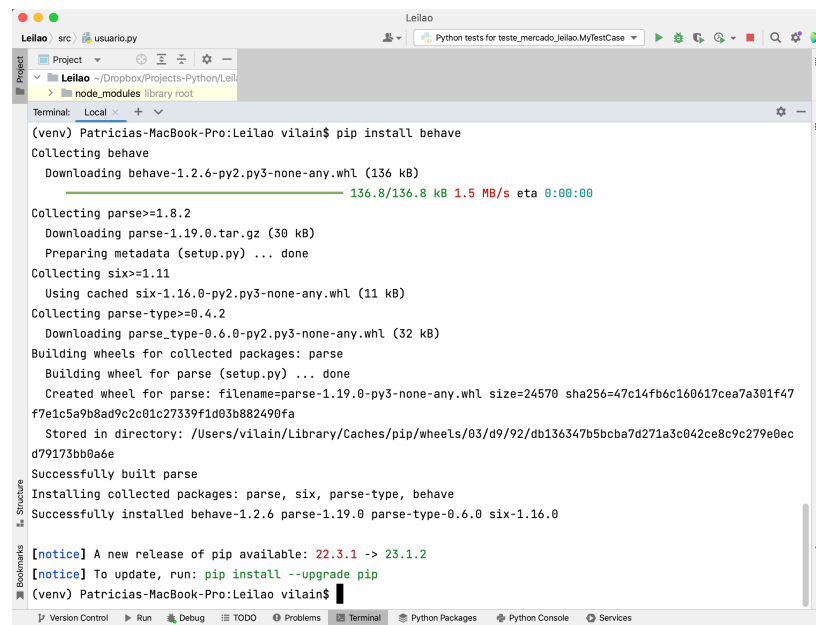


Figura 2 - Instalação do Behave completada.

2 CONFIGURANDO O PROJETO

Para executar os testes de aceitação utilizando o framework Behave, a seguinte estrutura de diretórios deve ser criada: dentro do diretório de testes, crie um diretório *features* e, dentro do diretório *features*, crie o diretório *steps* (Figura 3).

No diretório *features* serão incluídos os arquivos com extensão *.feature*, que contêm os testes de aceitação escritos com a linguagem Gherkin.

No diretório *steps* serão incluídos os arquivos com extensão *.py*, que contêm o *glue code* que fará a ligação entre os testes de aceitação e a aplicação, neste caso o mercado de leilão.

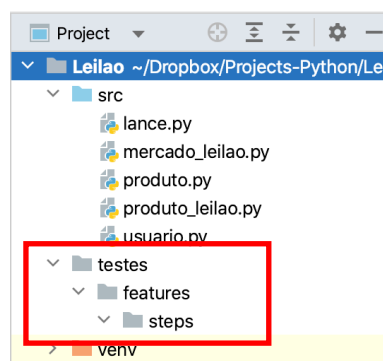


Figura 3 - Diretórios *features* e *steps*.

3 CRIANDO O PRIMEIRO TESTE

Os testes de aceitação utilizados pelo *Behave* são escritos em texto plano em uma linguagem de domínio chamada *Gherkin Language*. No decorrer deste estudo dirigido,

iremos conhecer a sintaxe desta linguagem e como os testes de aceitação escritos em Gherkin são ligados ao código-fonte.

Para criar seu primeiro teste, de dentro do diretório *features*, crie o arquivo com extensão *CadastrarUsuario_Test.feature* (Figuras 4 e 5). Note que a extensão deste arquivo é *.feature*.

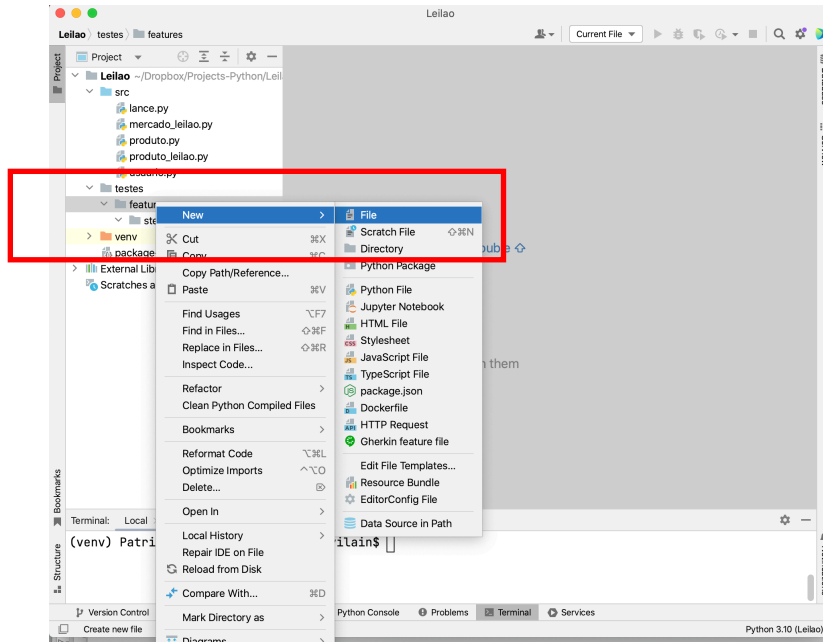


Figura 4 - Adicionando um arquivo no diretório *features*

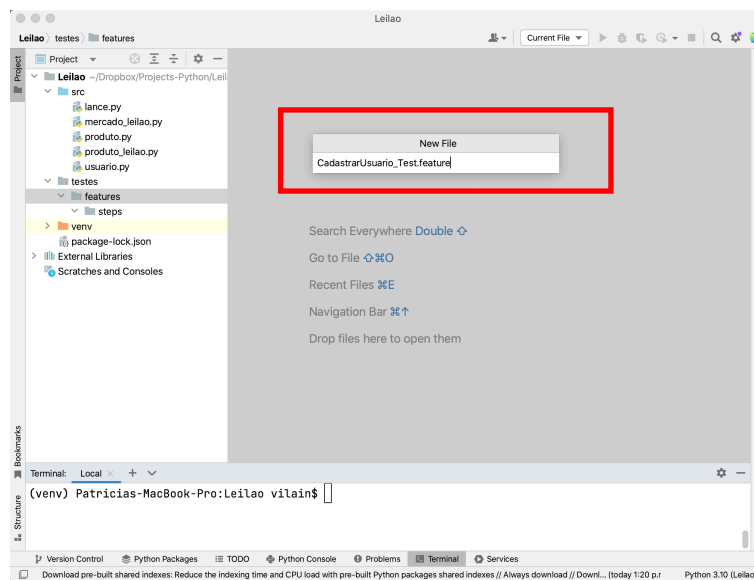


Figura 5 - Criando o arquivo *CadastrarUsuario_Test.feature*

Dentro do arquivo *CadastrarUsuario_Test.feature* inclua o teste de aceitação para cadastrar um usuário. Copie o conteúdo do quadro a seguir.

Quadro 1 - Código-fonte da feature para cadastrar usuário

```
Feature: Cadastrar Usuario
  Cadastrar usuarios

Scenario: Cadastrar Usuario com Sucesso
  Given O nome de usuario Ernani Cesar
    And o endereco Campus Universitario
    And o CPF 055.761.919-00
    And o e-mail ernani.santos@posgrad.ufsc.br
  When O usuario eh cadastrado
  Then O sistema deve possuir usuarios
```

A Figura 6 mostra o arquivo CadastrarUsuario_Test.feature com o nosso primeiro teste de aceitação. Note que o texto em amarelo representa os *steps* do teste de aceitação que ainda não tem um *glue code*. À medida que as funções correspondentes a cada *step* vão sendo implementadas, estes *steps* não vão mais estar em destaque.

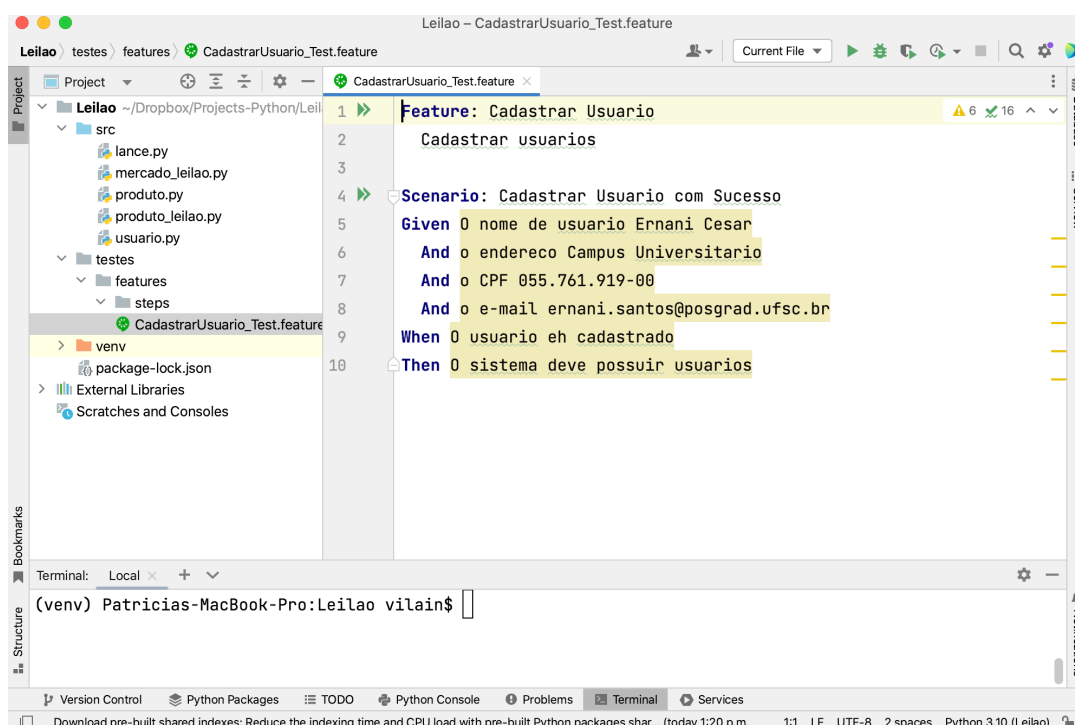


Figura 6 - Primeiro Teste de Aceitação.

Pronto! O primeiro teste foi criado, na próxima seção iremos criar o glue code necessário para executar este teste.

4 EXECUTANDO O TESTE PARA GERAR *TEMPLATE* DE *GLUE CODE*

Se você selecionar o arquivo CadastrarUsuario_Test.feature e pedir para executá-lo (Figura 7), você perceberá que o Behave avisa que o *glue code* correspondente aos *steps* desta *feature* não estão definidos, conforme mostra a Figura 8.

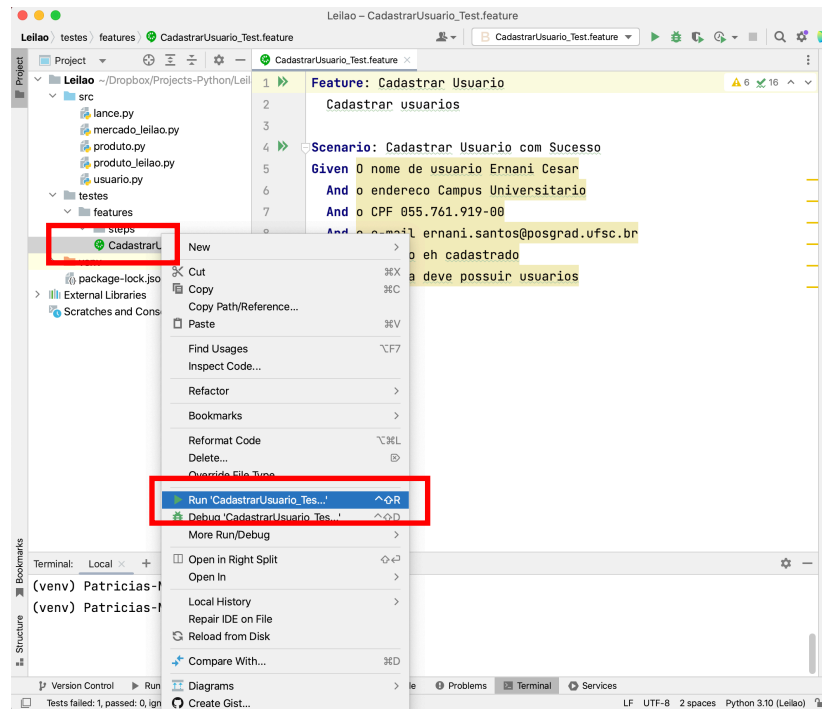


Figura 7 - Executando um teste de aceitação

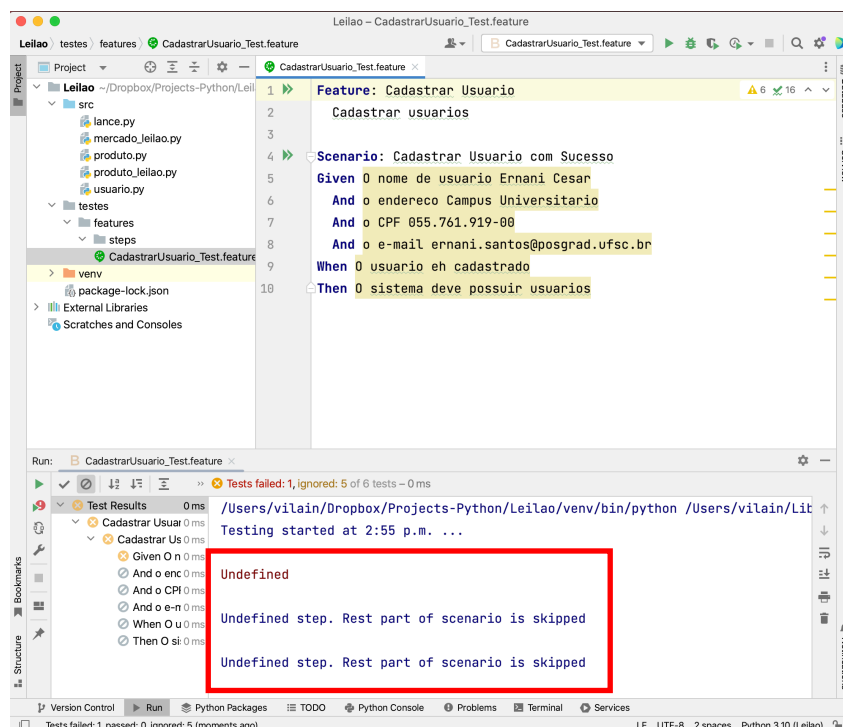


Figura 8 - Steps não definidos

É possível pedir para o *Behave* gerar o template com as funções correspondentes ao steps da feature (Figura 9). Para tanto, é necessário dizer em qual arquivo de steps estas funções serão incluídas (Figura 10). É possível renomear este arquivo, mas percebe-se que ele é incluído no diretório *steps*, porque é no diretório *steps* que o *Behave* irá procurar o glue code.

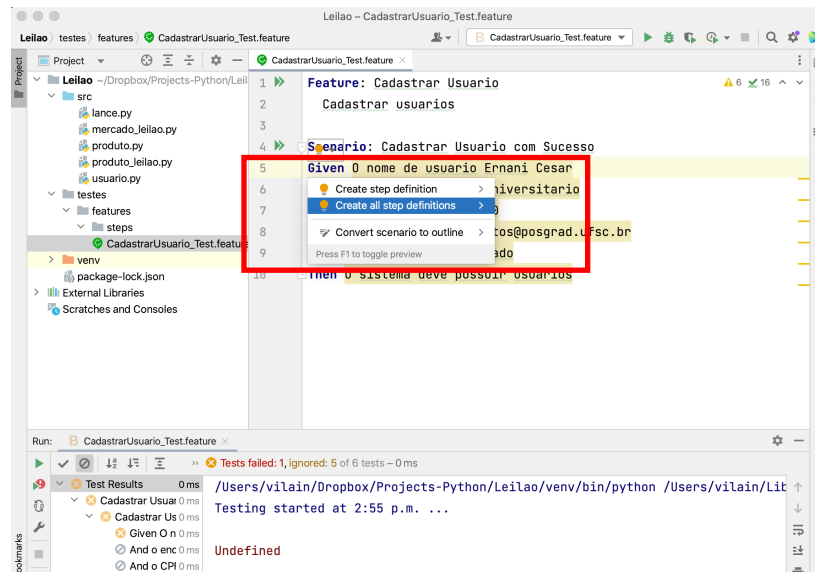


Figura 9 - Gerando as definições de cada step

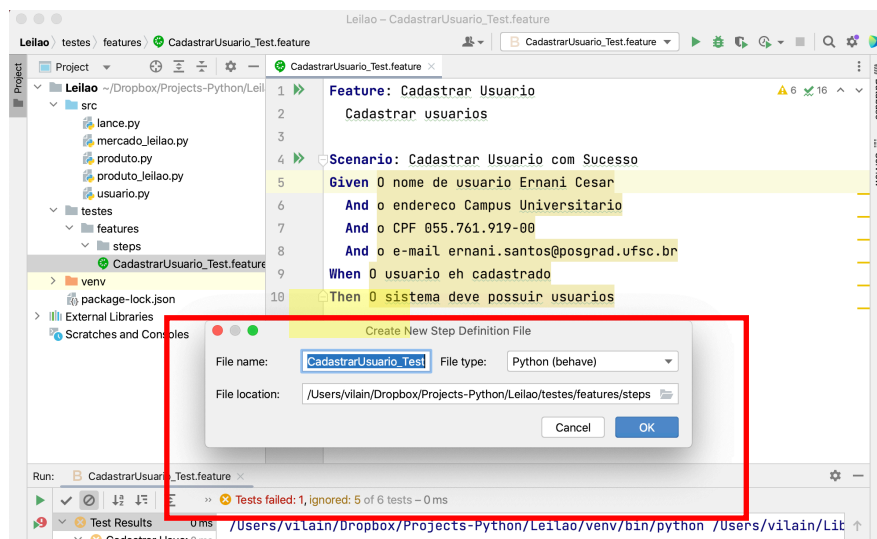


Figura 10 - Criando o arquivo com os steps

A Figura 11 mostra o template gerado automaticamente pelo Behave.

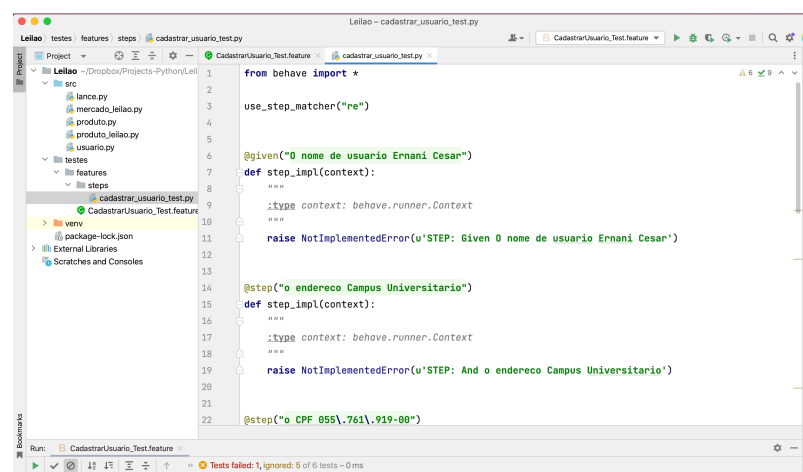


Figura 11 - Criando o arquivo com os steps

5 CRIANDO O *GLUE CODE* E EXECUTANDO O TESTE

Agora é necessário completar as funções do arquivo `cadastrar_usuario_test.py`. Para facilitar, copie o código-fonte disponibilizado no quadro a seguir para dentro do `cadastrar_usuario_test.py`.

Quadro 2 - Código-fonte do arquivo `cadastrar_usuario_test.py`

```
from behave import *
from mercado_leilao import MercadoLeilao

@given('O nome de usuario {nome_usuario}')
def step_impl(context, nome_usuario):
    context.nome_usuario = nome_usuario

@given('o endereco {endereco_usuario}')
def step_impl(context, endereco_usuario):
    context.endereco_usuario = endereco_usuario

@given('o CPF {cpf_usuario}')
def step_impl(context, cpf_usuario):
    context.cpf_usuario = cpf_usuario

@given('o e-mail {email_usuario}')
def step_impl(context, email_usuario):
    context.email_usuario = email_usuario

@when('O usuario eh cadastrado')
def step_impl(context):
    context.mercado = MercadoLeilao()
    context.mercado.cadastra_usuario(context.nome_usuario, context.endereco_usuario,
    context.email_usuario, context.cpf_usuario)

@then('O sistema deve possuir usuarios')
def step_impl(context):
    assert context.mercado.possui_usuario() == True
```

Finalmente, para executar este teste de aceitação, selecione a feature `CadastrarUsuario_Test.feature`, selecione o arquivo correspondente a esta *feature* e peça para executá-lo (Figura 12).

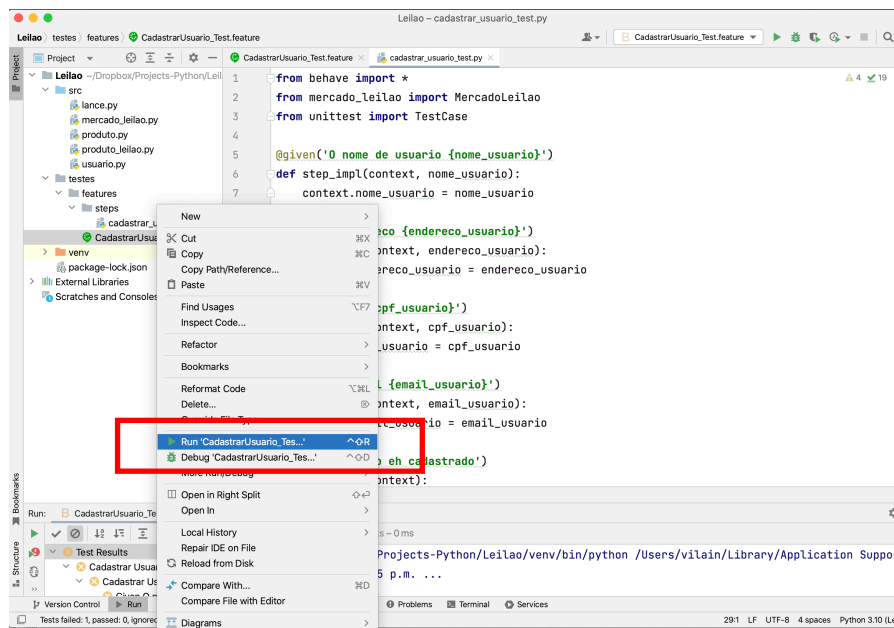


Figura 12 - Executando a *feature* correspondente ao primeiro teste de aceitação

A Figura 13 mostra que este teste de aceitação é executado com sucesso, sendo 6 testes executados, cada um correspondendo a um *step* da *feature*.

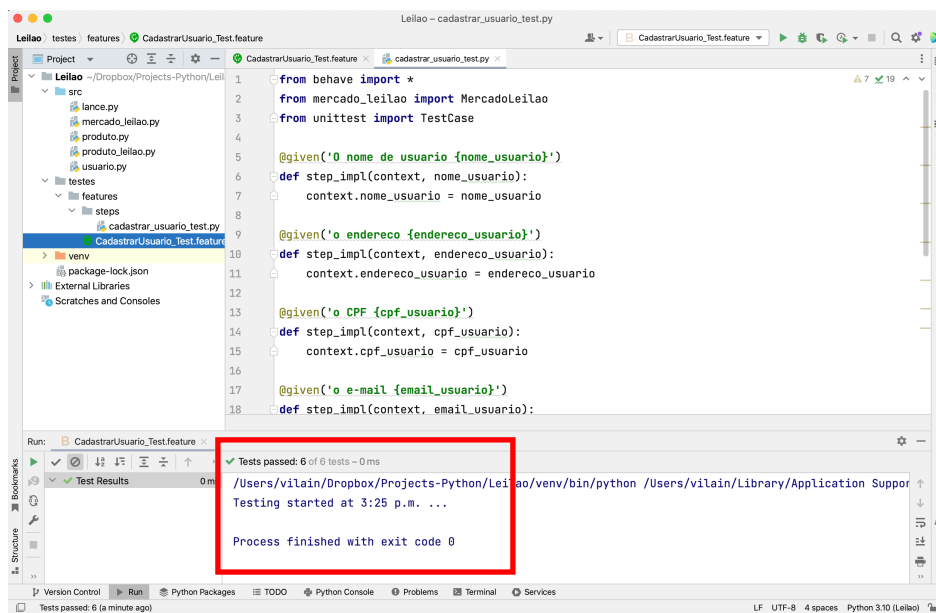


Figura 13 - Resultado da execução do primeiro teste de aceitação

Observe que o *Behave* possui um objeto *context* que permite que informações sejam trocadas entre diferentes *steps*. No teste de aceitação *CadastrarUsuario_Test.feature*, como seria possível que o nome do usuário fornecido no primeiro *step* fosse utilizado, mas adiante, no *step* que cadastra o usuário? Essa passagem de informações entre diferentes *steps* é feita através deste objeto *context*. No exemplo em questão, no primeiro *step* é criado um atributo chamado *nome_usuario*, que depois é utilizado no *step* que faz o cadastro deste usuário.

6 CRIANDO UM CENÁRIO DO TIPO OUTLINE

Muitas vezes um conjunto de cenários tem o mesmo comportamento, modificando somente os dados utilizados. Ao invés de definirmos diversos cenários semelhantes, é possível "parametrizar" um cenário com diversas entradas. Na linguagem Gherkin isto é feito através do Scenario Outline.

Vamos supor que, ao invés de cadastrar somente um usuário no sistema de mercado de leilão, queremos cadastrar diversos usuários. Como o cadastro de cada usuário vai ser realizado de forma semelhante, podemos definir um Scenario Outline que vai ser responsável pelo cadastro de todos os usuários. Cada linha da tabela associada ao Scenario Outline é vista como um cenário, ou seja, um teste de aceitação.

O Quadro 3 apresenta um exemplo de Scenario Outline para a funcionalidade (feature) de cadastro de usuários.

Quadro 3 - Scenario Outline para Cadastro de Usuários

Feature: Cadastrar Usuario					
Cadastrar usuarios					
Scenario Outline: Cadastrar Usuario com Sucesso					
Given o nome de usuario de <nome>					
And o CPF <cpf>					
And e o endereco <end>					
And e o e-mail <email>					
When cadastra o usuario					
Then o sistema retorna a mensagem <mensagem>					
Examples:					
nome	cpf	end	email	mensagem	
Ernani Cesar	055.761.919-00	UFSC	ernani@posgrad.ufsc.br	Usuario cadastrado com sucesso	
Ernani Cesar	055.761.919-99	UFSC	ernani@posgrad.ufsc.br	CPF invalido: 055.761.919-99	
Patricia	447.115.568-77	UFSC	patricia@posgrad.ufsc.br	Usuario cadastrado com sucesso	

De forma semelhante à criação da *feature* para fazer o cadastro de um usuário, crie uma *feature* para fazer o cadastro de um conjunto de usuários, seguindo o conteúdo do quadro acima. A Figura 14 mostra a *feature* CadastrarConjuntoUsuarios_Test.feature já criada no PyCharm.

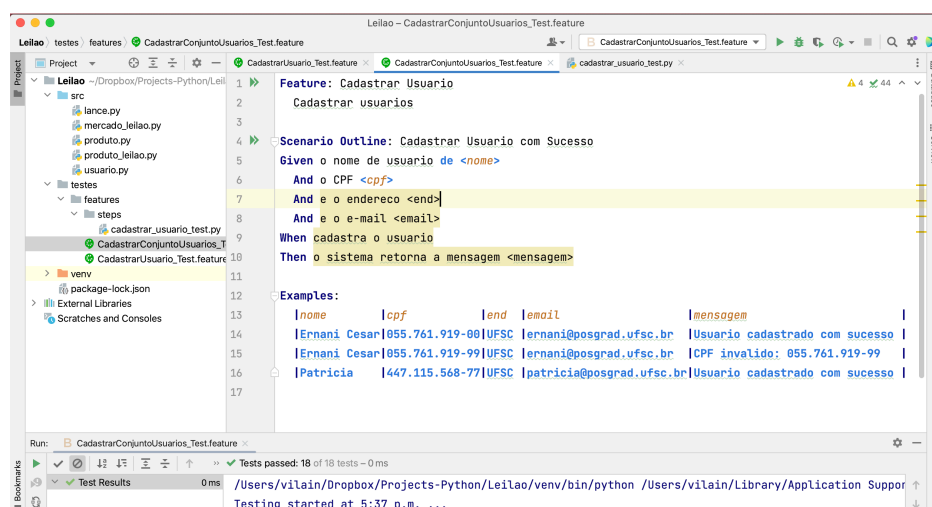


Figura 14 - Teste de Aceitação CadastrarConjuntoUsuarios_Test.feature

Propositalmente, o texto de alguns *steps* desta *feature* são iguais aos *steps* da *feature* anterior que fazia a inclusão de somente um usuário e outros não. Observe que os dois primeiros *steps* não estão destacados em amarelo. Isto acontece porque uma função com o mesmo nome já foi incluída anteriormente e será chamada quando estes *steps* forem executados. Para os *steps* nos quais os textos estão escritos de maneira diferente, o *Behave* irá chamar funções diferentes daquelas incluídas anteriormente. Como o código de cola correspondente aos *steps* com o mesmo texto também são iguais, isto não trará nenhum problema para a execução.

Na continuação deste novo teste de aceitação, crie o arquivo `cadastrar_conjunto_de_usuarios_test.py` dentro do diretório *steps*. Copie o código de cola apresentado no quadro a seguir para dentro deste arquivo. Observe que este código de cola não tem função para os dois primeiros *steps* pois elas já estão disponíveis no arquivo de *steps* criado anteriormente. Se definirmos novamente um código de cola para estes *steps*, o *Behave* vai apresentar um erro avisando que o código já foi definido.

Mais um teste de aceitação pronto! Agora é só executar a *feature* `CadastrarConjuntoUsuarios_Test.feature`.

Quadro 4 - Código-fonte do arquivo `cadastrar_conjunto_de_usuarios_test.py`

```
from behave import *
from mercado_leilao import MercadoLeilao

@given('e o endereço {end_usuario}')
def step_impl(context, end_usuario):
    context.endereco_usuario = end_usuario

@given('e o e-mail {email_usuario}')
def step_impl(context, email_usuario):
    context.email_usuario = email_usuario

@when('cadastra o usuario')
def step_impl(context):
    context.mercado = MercadoLeilao()
    try:
        context.mercado.cadastra_usuario(context.nome_usuario, context.endereco_usuario,
                                          context.email_usuario, context.cpf_usuario)
        context.msg = 'Usuario cadastrado com sucesso'
    except Exception as e:
        context.msg = e.__str__()

@then('o sistema retorna a mensagem {mensagem}')
def step_impl(context, mensagem):
    assert mensagem == context.msg
```



Exercício:

- 1) Faça um teste de aceitação para cadastrar produtos no mercado de leilão. Este teste deverá conter 2 cenários. No primeiro cenário, somente um produto é cadastrado no mercado de leilão. No segundo cenário, dois produtos são cadastrados com o mesmo nome. Neste segundo cenário o cadastro do primeiro produto ocorre normalmente, mas o cadastro do segundo produto com o mesmo nome deve retornar a mensagem "O produto ja existe ou o leiloador nao esta cadastrado." indicando que dois produtos com o mesmo nome não podem ser incluídos (mensagem gerada quando uma exceção é gerada).

Observações:

- Para cadastrar um produto é necessário primeiro ter um usuário cadastrado, portanto, utilize a *keyword* **Background**, para incluir o que precisa ser feito antes de cada cenário, ou seja, a criação de um mercado de leilão e o cadastro de um usuário. Desta forma, o método correspondente ao Background é executado antes de cada cenário.
- Datas não estão sendo tratadas neste programa, portanto, qualquer número pode ser utilizado como data.
- Cada feature deve estar em um arquivo separado. Portanto, a feature deste exercício deve estar em um arquivo separado da feature que foi implementada durante o estudo dirigido.

Feature: Cadastrar Produto Usando Setup
Cadastrar usuarios

Background:

Given o cadastro do usuario Ernani Cesar foi realizado

Scenario: Cadastrar Produto com Sucesso

Given o nome do produto sofa

And a descricao do produto amarelo

And e o lance 100

And e o cpf do leiloador 055.761.919-00

When cadastrar o produto

Then o sistema cadastra com sucesso

Scenario: Cadastro de Produto com Problema

Given sofa amarelo ja foi cadastrado

And o nome do produto sofa

And a descricao do produto amarelo

And e o lance 100

And e o cpf do leiloador 055.761.919-00

When cadastrar o produto

Then o sistema mostra a mensagem O produto ja existe ou o leiloador nao esta cadastrado.